

Diagnóstico de Ferramental e Confronto de Paradigmas

Mini-caso: Sistema de Empréstimo de Equipamentos do Laboratório

Nome dos integrantes: Murilo Pacheco - 213390 e Italo Silva - 077930

Premissas adotadas:

- Cada equipamento possui um código único de identificação.
- Alunos e professores possuem matrícula ou identificação institucional.
- O sistema registra a data efetiva de devolução.
- Um equipamento só pode ser emprestado quando estiver disponível.
- O bloqueio de 15 dias começa a contar a partir da devolução atrasada.
- Um novo empréstimo só pode ser realizado quando o solicitante não possuir bloqueio ativo.

INVENTÁRIO E CLASSIFICAÇÃO DO FERRAMENTAL

1. Levantamento e classificação das ferramentas

- **diagrams.net / draw.io**
 - Versão: 30.0.0
 - Categoria CASE: Upper CASE
 - Tipo funcional: Diagramação
 - Etapa do ciclo de vida: Análise e projeto
- **Visual Paradigm**
 - Versão: 18.0
 - Categoria CASE: Integrated CASE
 - Tipo funcional: Modelagem
 - Etapa do ciclo de vida: Requisitos, análise e projeto
- **MySQL Workbench**
 - Versão: 8.0.47
 - Categoria CASE: Lower CASE
 - Tipo funcional: Modelagem de dados
 - Etapa do ciclo de vida: Projeto e implementação do banco de dados
- **Visual Studio Code**
 - Versão: 1.131

- Categoria CASE: Lower CASE
- Tipo funcional: Codificação
- Etapa do ciclo de vida: Implementação e manutenção
- **Postman**
 - Versão: 12.23.0
 - Categoria CASE: Lower CASE
 - Tipo funcional: Teste
 - Etapa do ciclo de vida: Testes e validação de APIs
- **GitHub Desktop**
 - Versão: 3.5.11
 - Categoria CASE: Lower CASE
 - Tipo funcional: Repositório / controle de versão
 - Etapa do ciclo de vida: Implementação, manutenção e gerenciamento de configuração

2. Toolchain escolhido

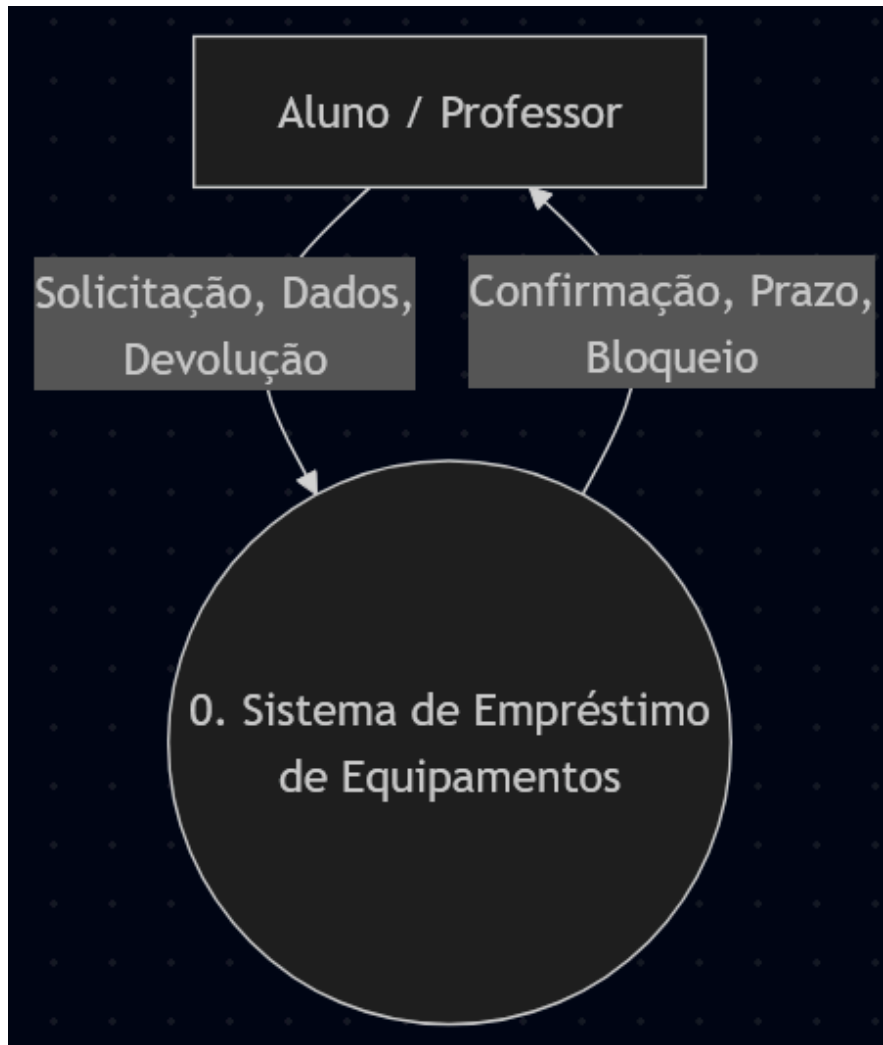
Para o desenvolvimento do Sistema de Empréstimo de Equipamentos do Laboratório foram escolhidos: **Visual Paradigm + Visual Studio Code + GitHub Desktop**.

- **Visual Paradigm:** Escolhido para a modelagem do sistema. **Benefício:** Representa visualmente classes e processos na mesma ferramenta, facilitando a compreensão da equipe. **Risco:** Dedicção de tempo excessivo à documentação ou o risco de os diagramas ficarem desatualizados em relação ao código.
- **Visual Studio Code:** Escolhido como ambiente principal de codificação. **Benefício:** Centraliza o desenvolvimento com suporte a múltiplas extensões, aumentando a produtividade. **Risco:** Uso excessivo de extensões gerando incompatibilidades ou diferenças entre os ambientes dos desenvolvedores.
- **GitHub Desktop:** Escolhido para o controle de versão Git. **Benefício:** Facilita o registro histórico e a colaboração no código. **Risco:** Conflitos de *merge* caso os integrantes editem os mesmos arquivos simultaneamente, exigindo comunicação constante e commits frequentes.

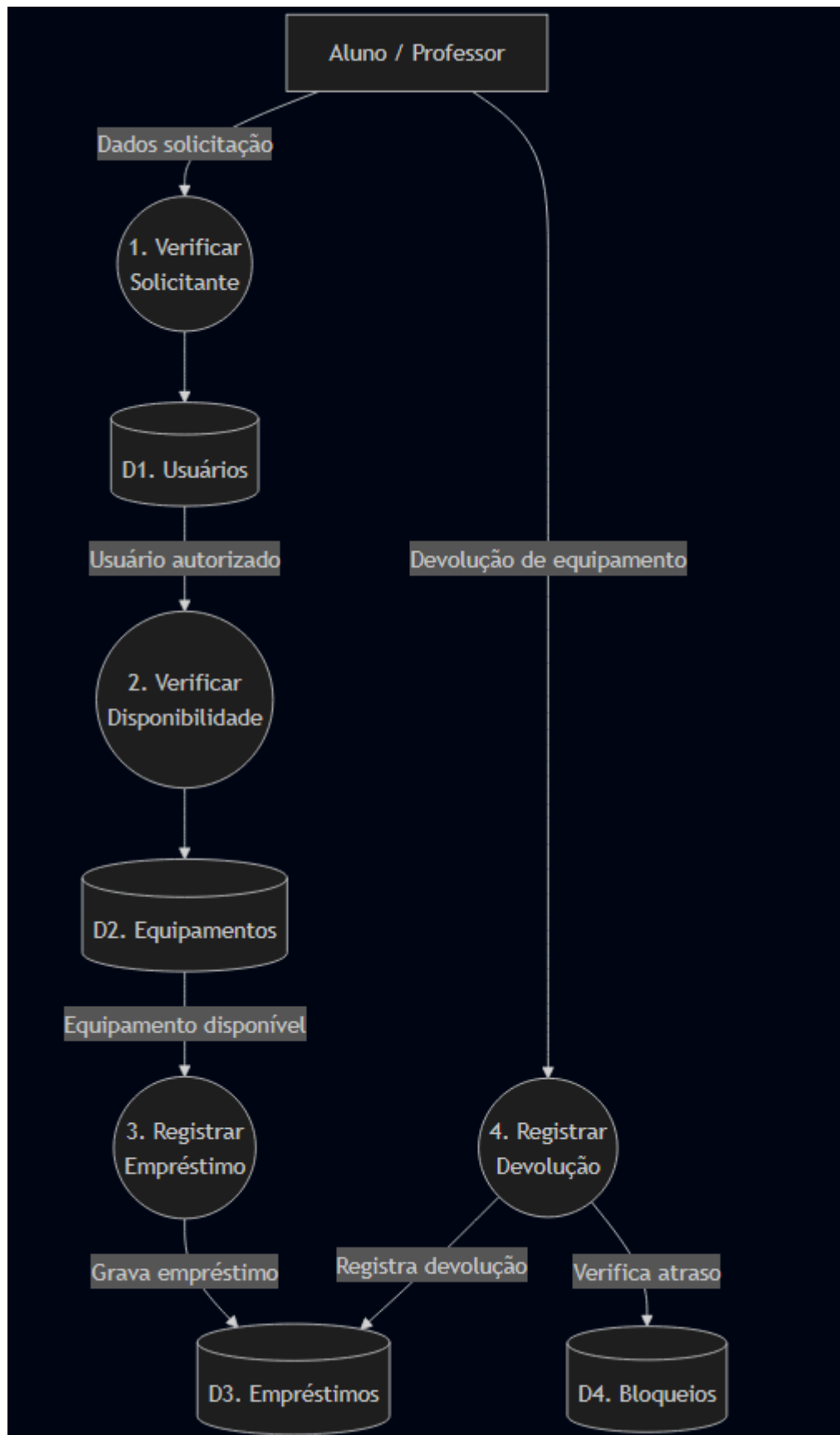
MODELAR O MESMO CASO NAS DUAS VISÕES

1. Visão Estruturada

1.1 DFD — Nível 0



1.2 DFD — Nível 1

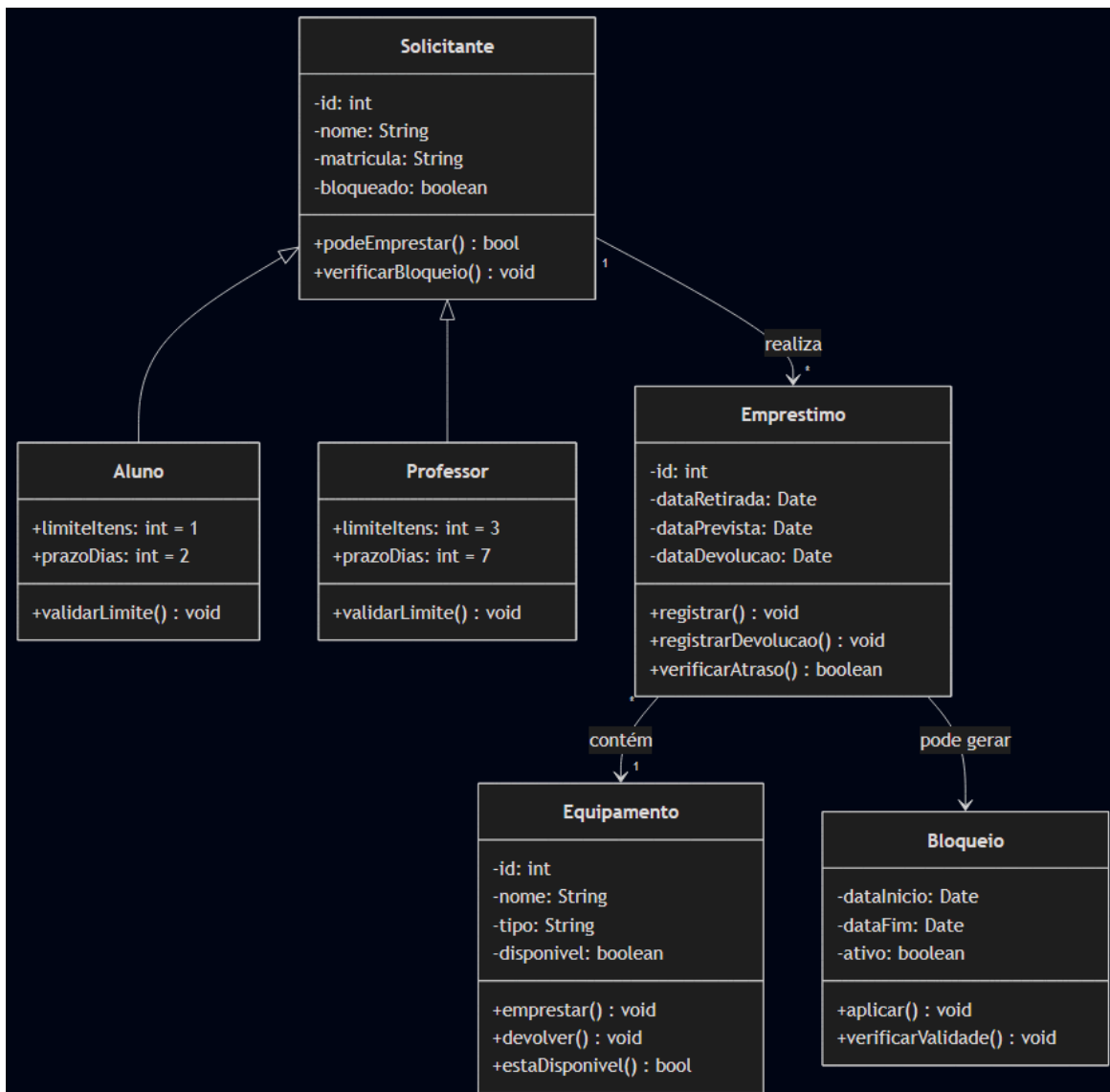


1.3 Dicionário de dados

- **idSolicitante:** Inteiro | Número positivo e único
- **nomeSolicitante:** Texto | Nome do aluno ou professor
- **tipoSolicitante:** Enum | ALUNO ou PROFESSOR
- **matricula:** Texto | Identificação institucional
- **idEquipamento:** Inteiro | Número positivo e único
- **nomeEquipamento:** Texto | Nome ou identificação do equipamento
- **tipoEquipamento:** Enum | NOTEBOOK, PROJETOR ou KIT_ROBOTICA
- **statusEquipamento:** Enum | DISPONIVEL ou EMPRESTADO
- **idEmprestimo:** Inteiro | Identificador único
- **dataRetirada:** Data | Data em que o equipamento foi retirado
- **dataPrevistaDevolucao:** Data | Data limite para devolução
- **dataDevolucao:** Data | Data efetiva de devolução
- **quantidadeEmprestimosAtivos:** Inteiro | Aluno: 0 a 1; Professor: 0 a 3
- **prazoMaximo:** Inteiro | Aluno: 2 dias; Professor: 7 dias
- **atraso:** Booleano | VERDADEIRO ou FALSO
- **dataInicioBloqueio:** Data | Data de início da penalidade
- **dataFimBloqueio:** Data | 15 dias após início do bloqueio
- **statusBloqueio:** Booleano | ATIVO ou INATIVO

2. Visão Orientada a Objetos

2.1 Diagrama de classes



2.2 Encapsulamento e Herança

- **Encapsulamento:** Aparece na proteção dos atributos internos das classes. Em Equipamento, o atributo - disponível é privado (indicado pelo sinal negativo). Ele não é alterado diretamente por fora do sistema, mas sim através de métodos públicos como + emprestar() e + devolver(), garantindo que a própria classe controle seu estado.
- **Herança:** Utilizada na relação entre a classe base Solicitante e as subclasses Aluno e Professor. Atributos comuns (id, nome, matricula) ficam na superclasse, evitando repetição, enquanto cada subclasse mantém apenas suas regras específicas de negócio (limites de itens e prazos)

CONFRONTO DOS DOIS MODELOS

Os dois paradigmas são úteis, porém analisam o problema de perspectivas diferentes. Avaliando sob quatro critérios principais:

1. **Foco da modelagem:** O DFD foca no fluxo da informação (verificar solicitante, registrar empréstimo), enquanto a orientação a objetos foca nas entidades (Aluno, Equipamento).
2. **Reutilização:** O modelo estruturado reaproveita processos, mas não possui mecanismos como a herança. A OO permite que Aluno e Professor herdem de Solicitante, evitando duplicação de código.
3. **Manutenção:** Na OO, alterações em regras de negócios (ex: alterar o prazo de dias do aluno) são feitas em um único lugar (dentro da classe específica). No estruturado, a regra está espalhada pelos processos de verificação.
4. **Representação do domínio:** As classes espelham exatamente os agentes físicos do laboratório.

Conclusão: Para a implementação e manutenção deste sistema, a Orientação a Objetos atende melhor ao caso proposto. A utilização de classes e herança reduz duplicações, e o encapsulamento protege as regras de penalidades e prazos, tornando o software mais fiel ao cenário real.